

Modelagem dos Elementos de Projeto de Software

Parte 1

Profa. Dra. Lina Garcés



Nas aulas
anteriores ...

Paradigma de Desenvolvimento de Software OO

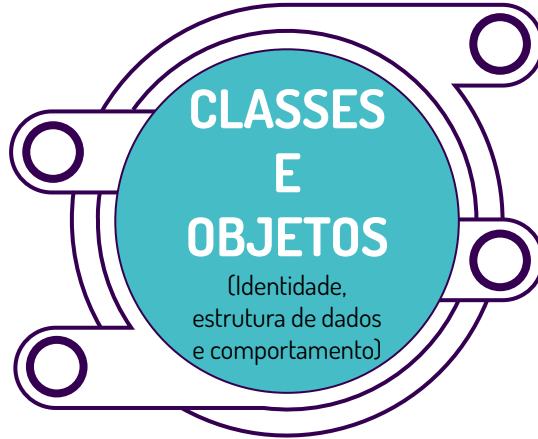
PRINCÍPIOS da OO

Abstração

Módulos, componentes, classes.

Encapsulamento

Ocultamento de informações e operações, Facilidade de mudança



Herança

Reuso de estrutura de dados e de comportamento . Coesão.

Polimorfismo

Flexibilidade na definição de métodos. Facilidade de evolução (incremento)

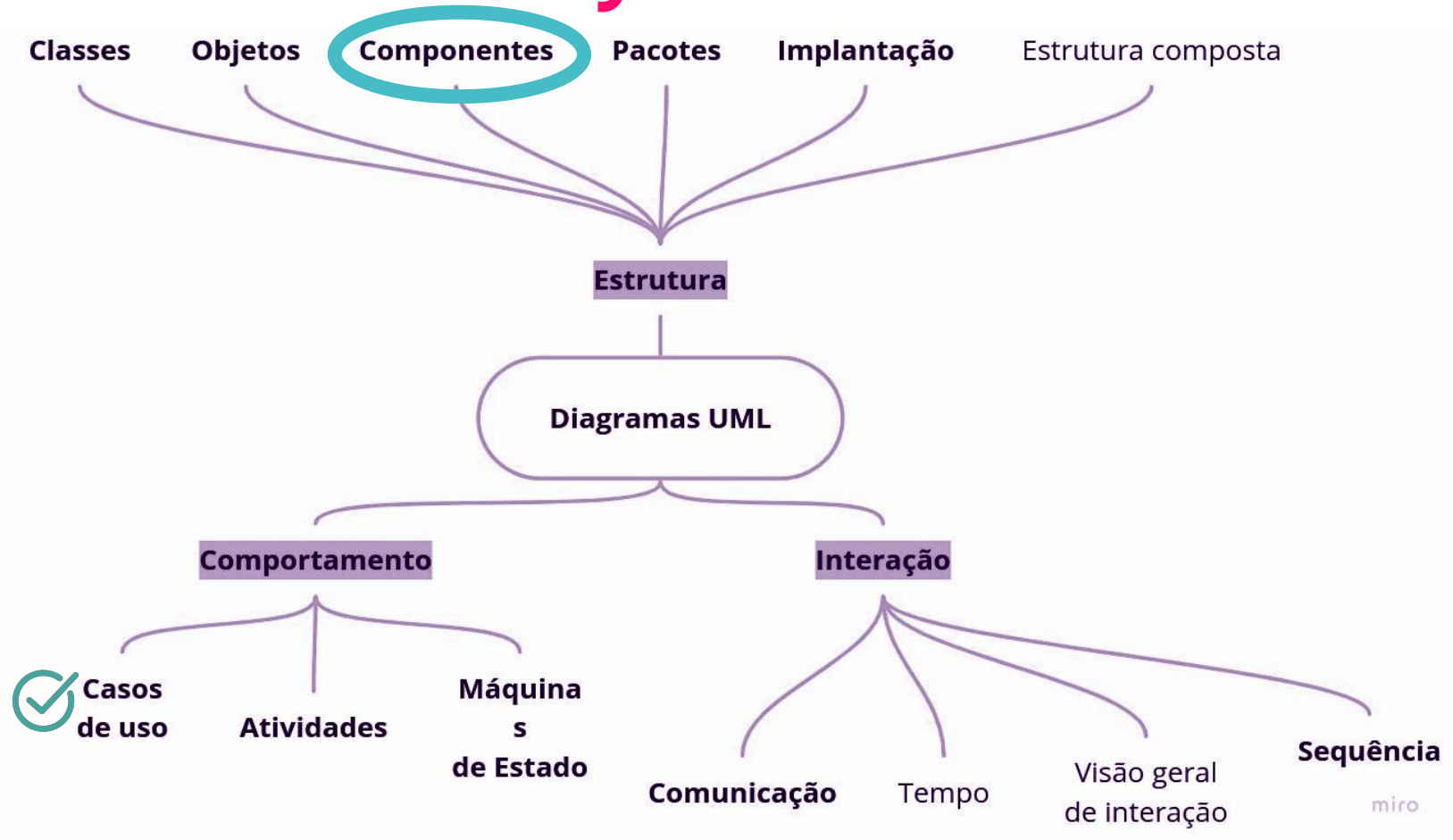
PRINCÍPIOS DE *DESIGN*

Baixo Acoplamento e Alta Coesão

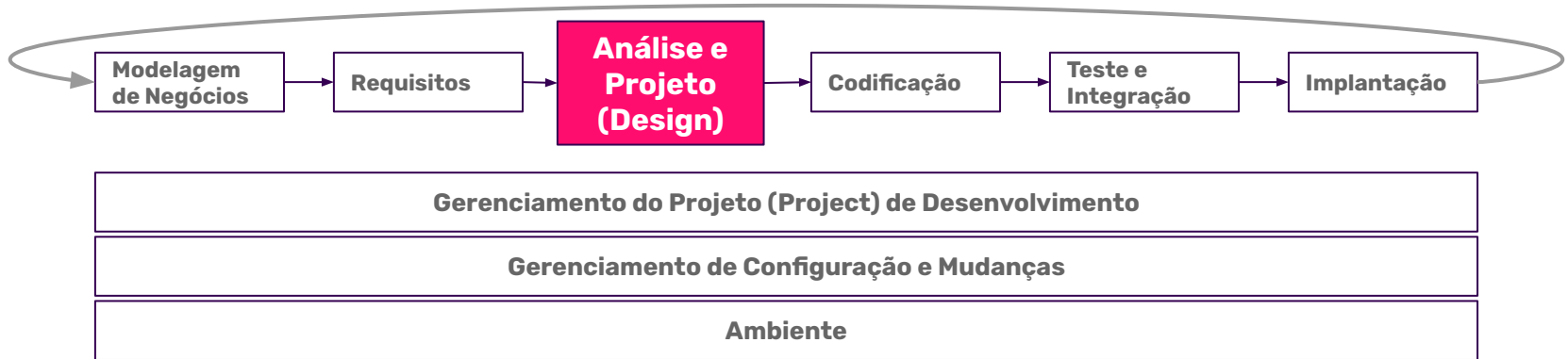
MODELAGEM - Diagramas UML (Unified Modeling Language)

PROCESSO DE DESENVOLVIMENTO DE SOFTWARE - Etapas de Análise, Design e Implementação

Modelagem com UML



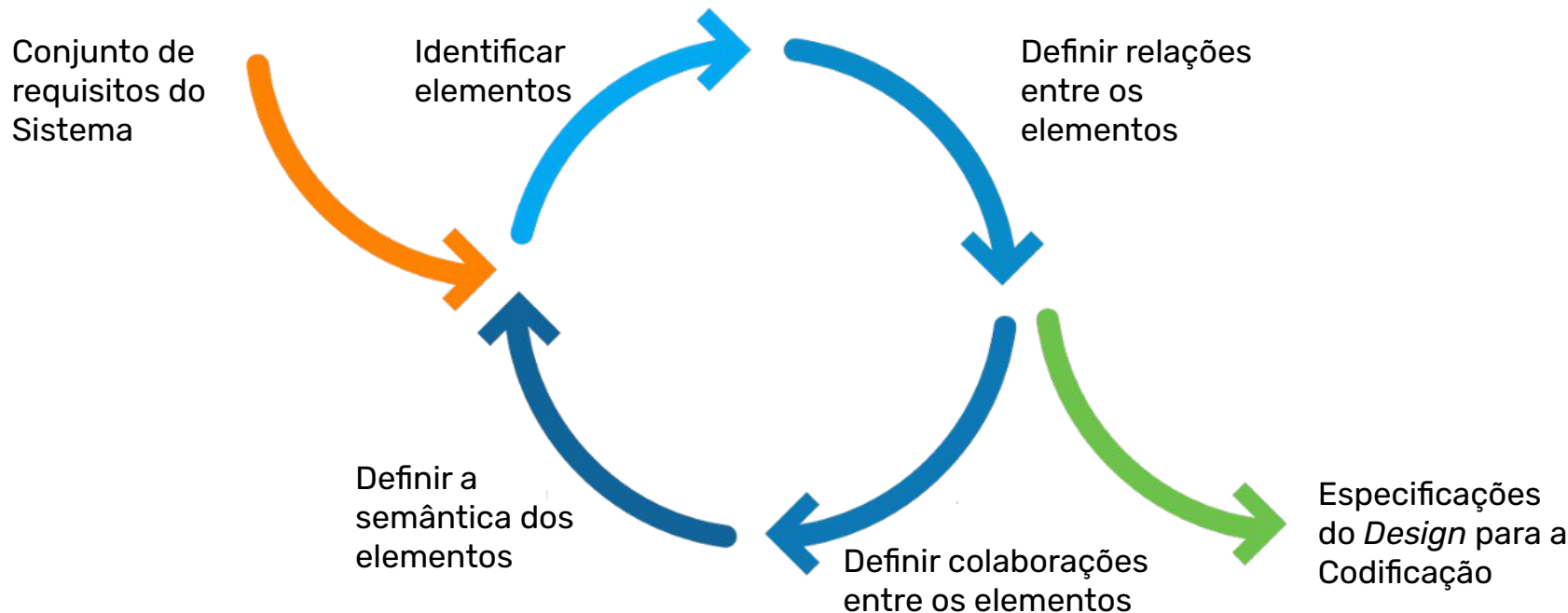
O Subprocesso de Análise e Projeto (Design)



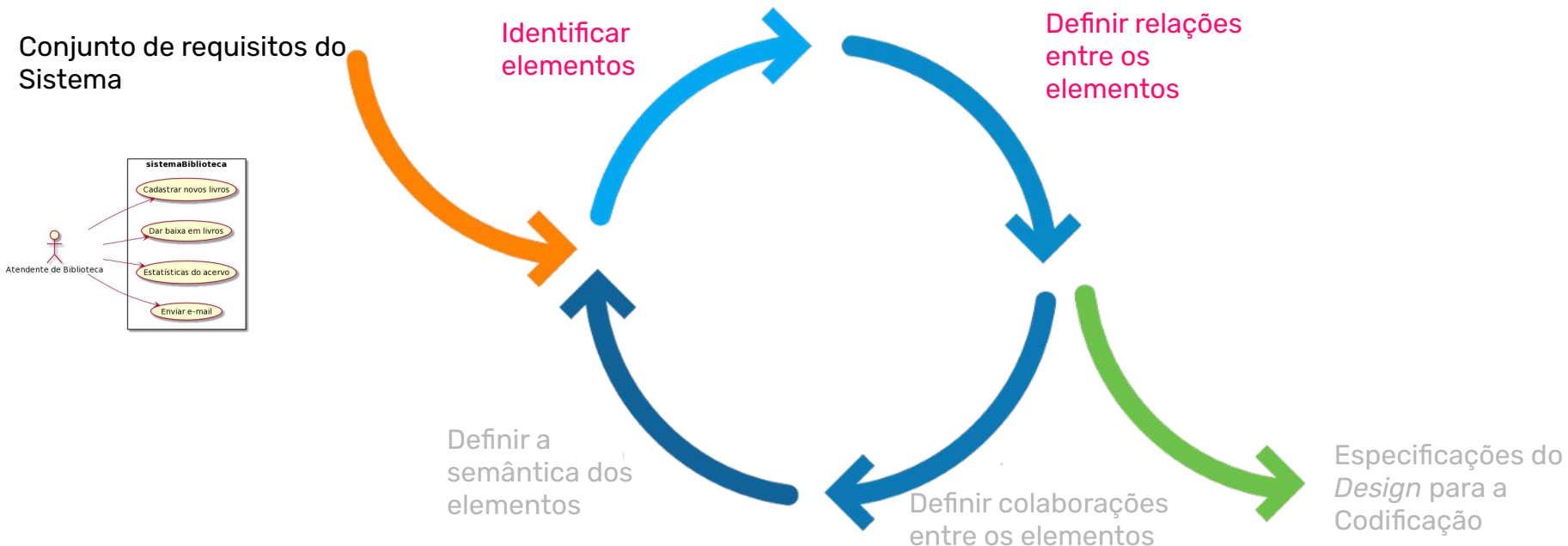
Objetivo da Análise e do Projeto

- **Transformar os requisitos em projeto (design) do sistema de software.**
- **Identificar os elementos (componentes, módulos, serviços, classes, objetos) que devem ser implementados no sistema**
- Criar uma arquitetura robusta do sistema considerando os elementos identificados
- Especificar a implementação dos elementos do sistema.

O Subprocesso de Análise e Projeto (Design)



O Subprocesso de Análise e Projeto (Design)



O que aprenderá na aula?

01

Identificação dos
elementos de projeto
de software

02

Identificação dos
Elementos no nível de
projeto da Arquitetura

03

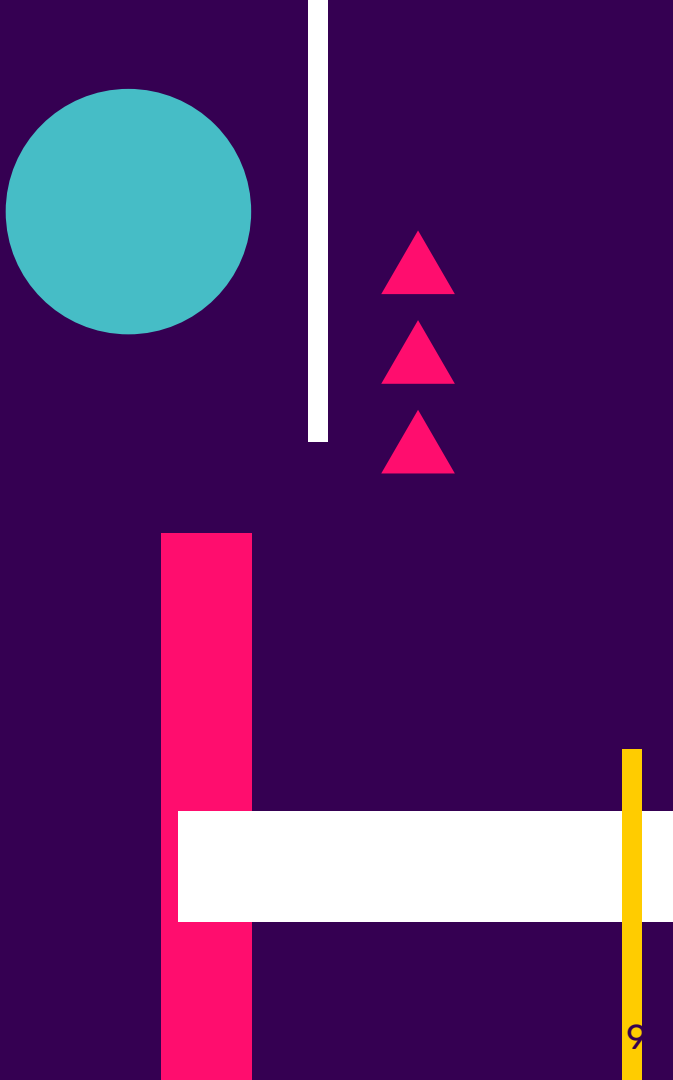
Identificação dos
elementos no nível de
projeto de
componentes

04

Modelo de classes
do domínio do
sistema

05

Resumo



01

Identificação dos Elementos de Projeto de Software

Abstrações nos Elementos de Projeto

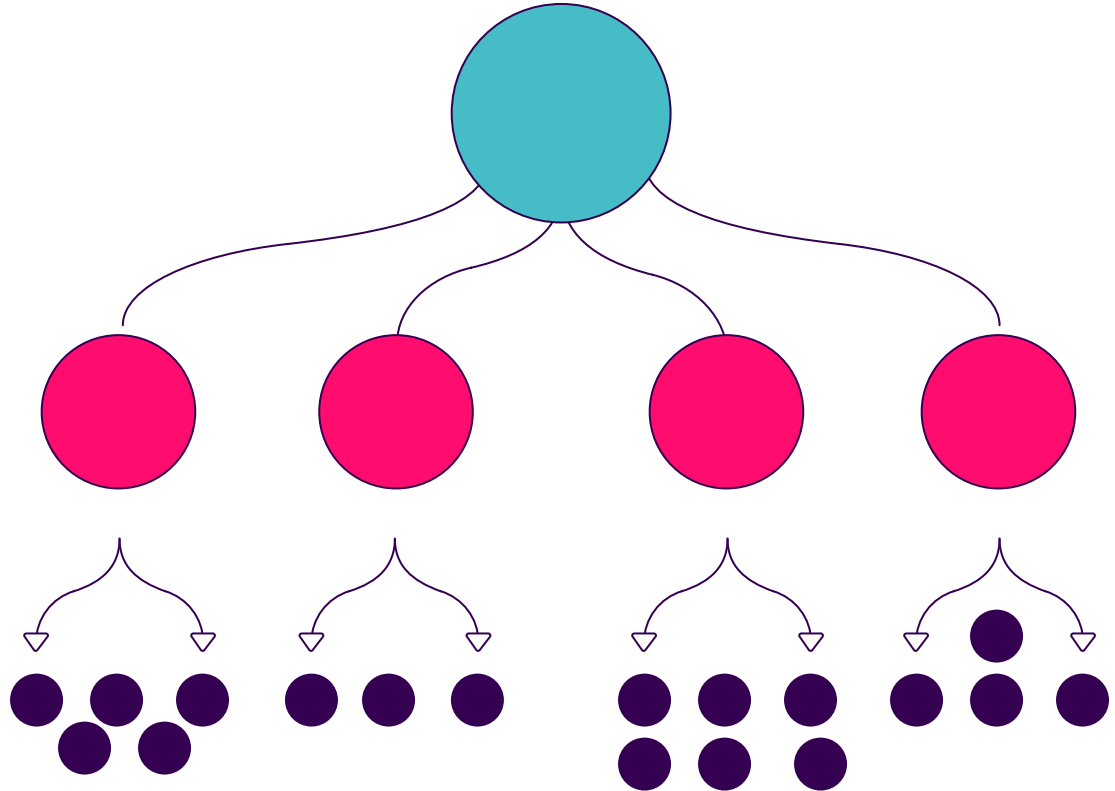
Sistema

Componentes

*(Nível de Arquitetura -
Projeto Abstrato)*

Classes e objetos

*(Nível interno de Componente -
Projeto Concreto)*



Identificação de Elementos de Projeto

Atividade que visa a **identificação, invenção, classificação e modelagem** das partes internas do sistema de software em um nível específico de abstração:

- **Nível arquitetural** → elementos são mais abstratos (componentes, módulos, serviços, ...)
- **Nível de componente** → elementos mais concretos e próximos à codificação, p.ex: classes, objetos, atributos, métodos



Classificação de Elementos de Projeto

Classificação é uma forma de organizar o conhecimento:

- Do problema que o sistema deve resolver
- Do domínio (conceitos, sinônimos, definições, linguagem de domínio)



Classificação de Elementos de Projeto

No projeto orientado a objetos é importante **reconhecer as semelhanças** entre os elementos do sistema

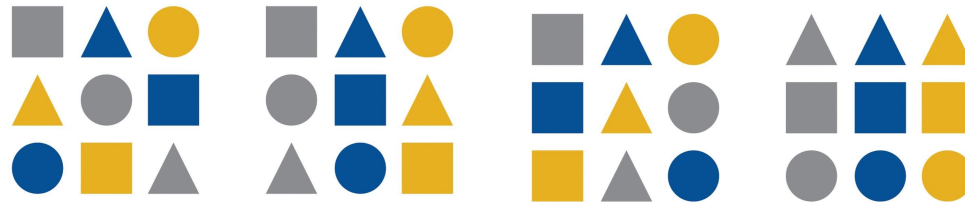
A identificação de **elementos comuns** permite definir soluções de menor tamanho e arquiteturas mais simples de entender

Um **projeto de software simples** é desejável em processos ágeis (XP, SCRUM...)

- Requerem de muito esforço e conhecimento da parte do arquiteto →
“Experiência é muito importante”
- É um processo incremental e iterativo

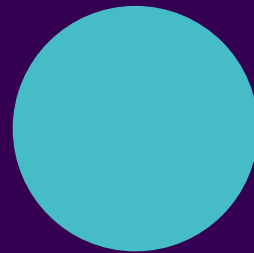
Classificação de Elementos de Projeto

- O tipo de classificação **depende dos motivos** para a organização dos elementos
- Não existe uma classificação “exata/perfeita”, mas algumas classificações são mais **adequadas** para o seu propósito.
- Classificações são **relativas à perspectiva** do profissional que a realiza
- Quanto **mais conhecimento** do domínio/problema (p.ex. Atividades executadas numa biblioteca) **mais** fácil de alcançar uma classificação adequada para o domínio/problema



02

Identificação dos Elementos de Projeto no Nível da Arquitetura



Identificação de Elementos de Projeto da Arquitetura

Possíveis elementos arquiteturais:

- Sistemas externos, bibliotecas, frameworks, componentes, módulos, serviços, APIs
- Dispositivos, sensores
- Middlewares, barramentos, brokers
- Repositórios, base de dados
- Serviços da Cloud (IaaS, PaaS, SaaS)

Identificação de Elementos de Projeto da Arquitetura

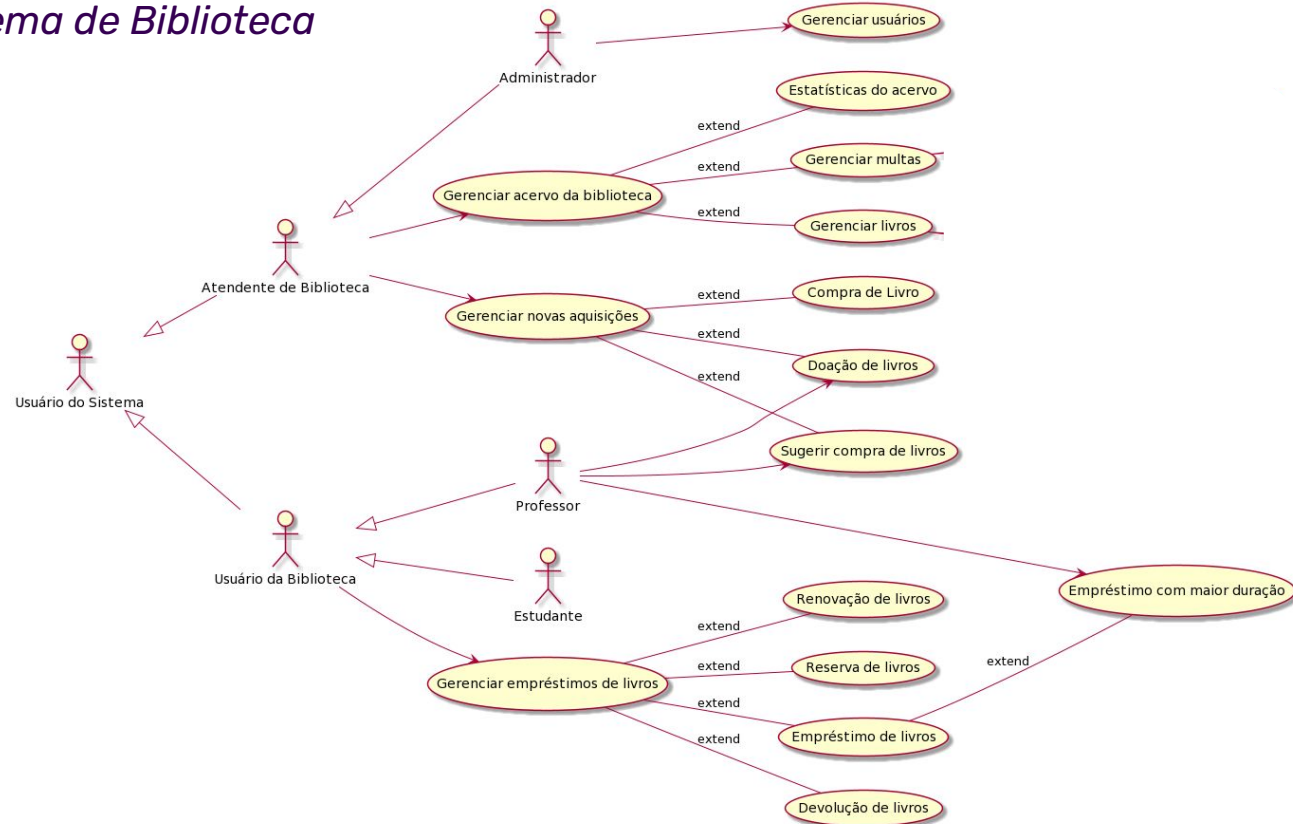
Classificação de Elementos com Base nas Responsabilidades

As responsabilidades de um elemento são o conhecimento do domínio (dados, informações) que ele administra ou gerencia, e as funcionalidades, capacidades e/ou serviços que oferece aos outros elementos do sistema.



Identificação de Elementos de Projeto da Arquitetura

Exemplo dos UC do Sistema de Biblioteca



Seleção dos componentes aplicando o princípio de alta coesão ou responsabilidade única ...

Identificação de Elementos de Projeto da Arquitetura

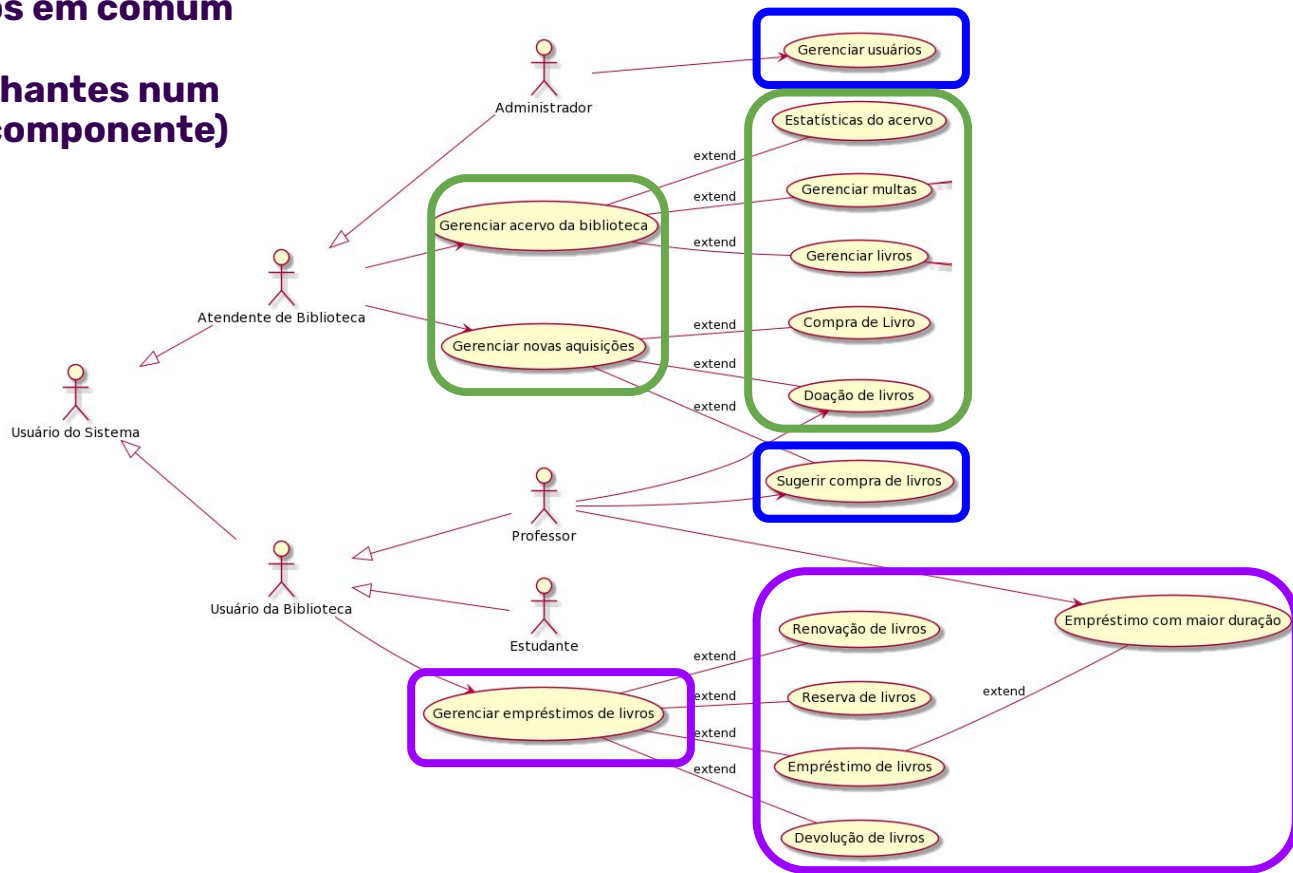
Grupos de UC com objetivos em comum

Alocar grupos de UC semelhantes num elemento da arquitetura (componente)

Componente de usuários

Componente de acervo

Componente de empréstimo

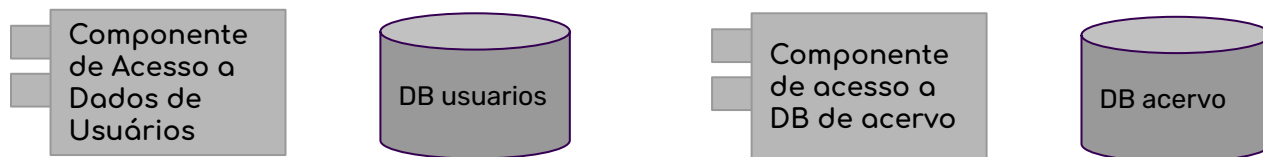


Identificação de Elementos de Projeto da Arquitetura

Identificação de Elementos Implícitos

Elementos que são **necessários** para cumprir com os requisitos do sistema, mas que **não foram identificados explicitamente nos UC.**

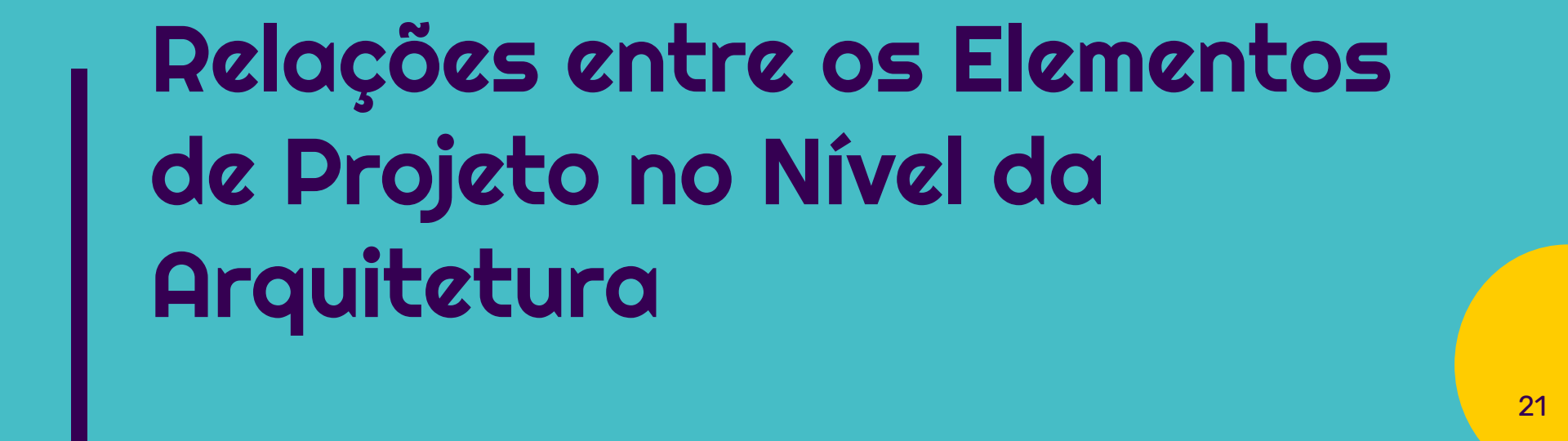
Exemplos: Outros sistemas, bibliotecas de software, frameworks, Dispositivos, sensores, Middlewares, barramentos, brokers, Repositórios, base de dados, Serviços da Cloud (IaaS, PaaS, SaaS)



⚠ *No caso do sistema de bibliotecas, os dados de usuários são gerenciados de forma centralizada pois é comum para todos os sistemas da USP*



03



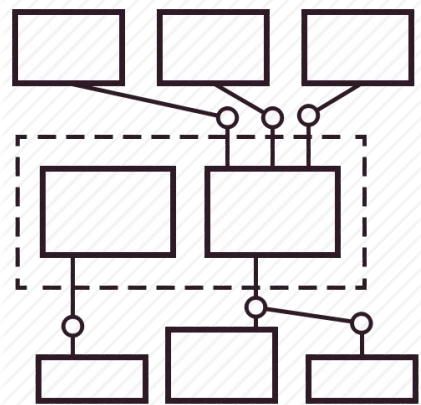
Relações entre os Elementos de Projeto no Nível da Arquitetura

Relacionamentos entre Elementos de Projeto da Arquitetura

No nível da arquitetura as relações entre elementos contribuem para

- entender a “estrutura” do sistema
- formar a base para o resto de relações entre outros elementos do sistema

As relações ajudam a definir os limites de cada elemento e a identificar quais elementos precisam colaborar.



Relacionamento do tipo <<Utiliza>>

Um elemento <<utiliza>> funções, serviços, dados ... de outro elemento

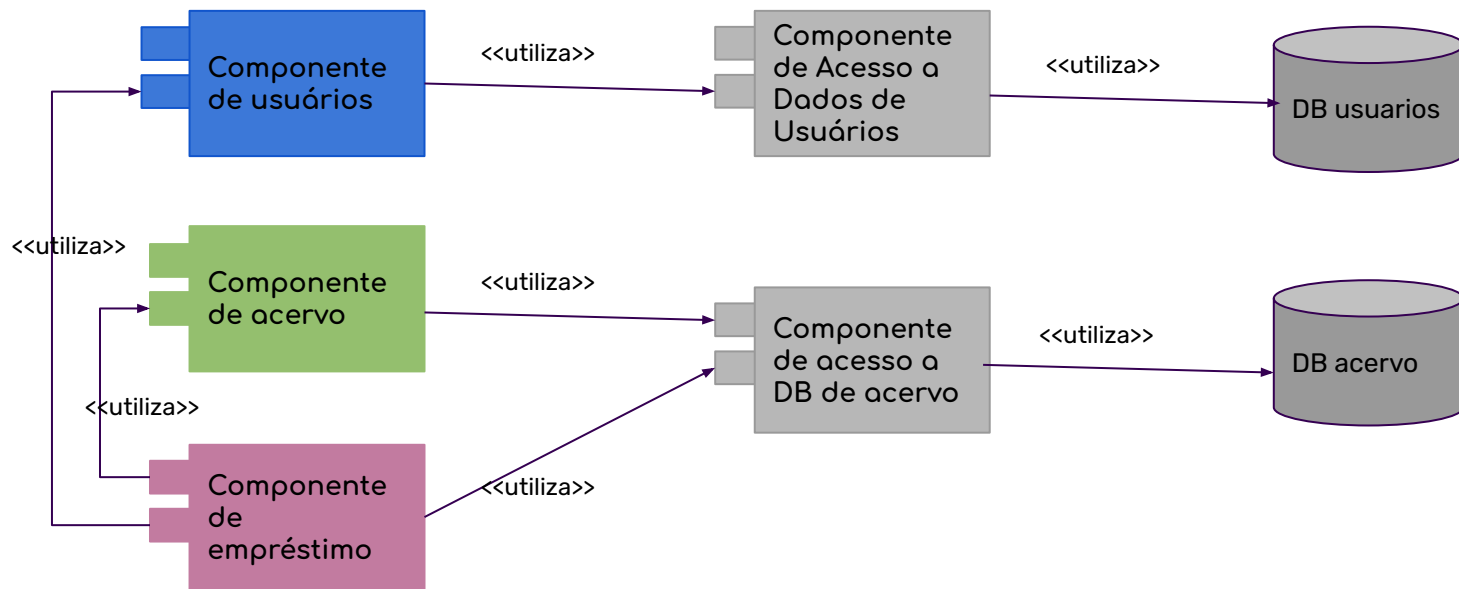
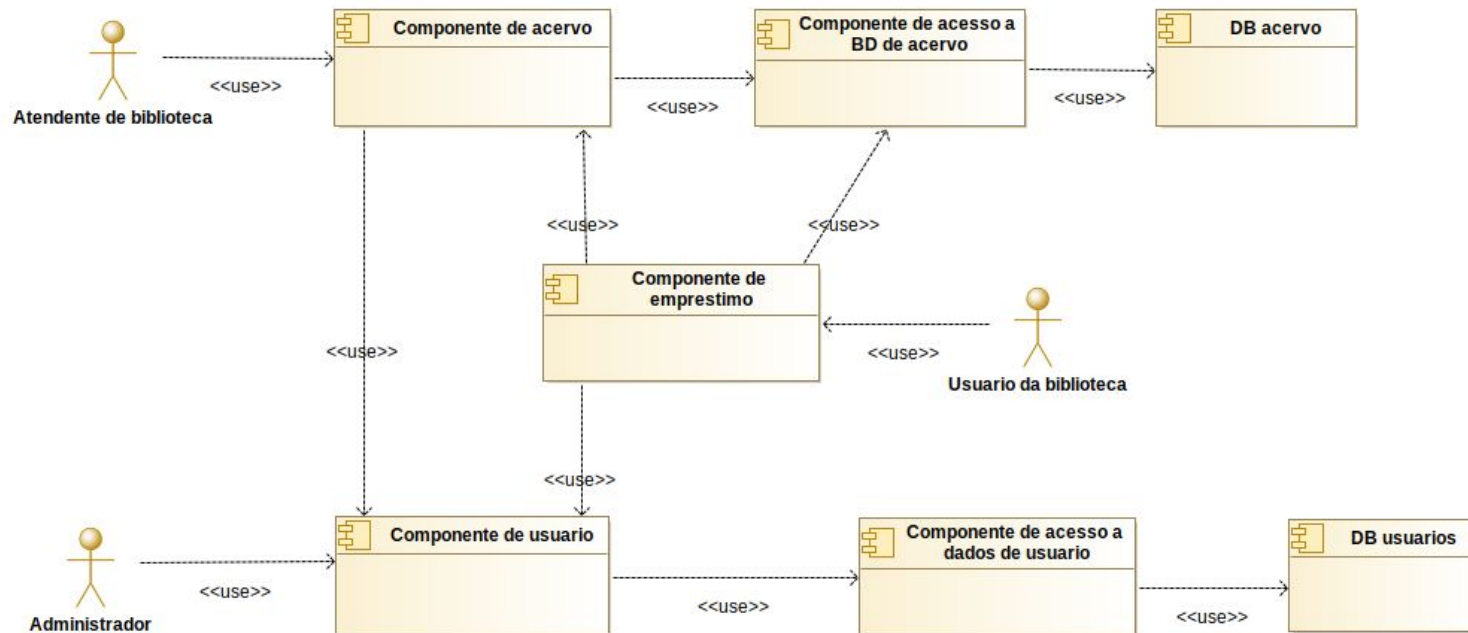


Diagrama de Componentes na UML

Um elemento <<utiliza>> funções, serviços, dados ... de outro elemento



Identificação e aplicação de padrões de estrutura entre elementos

Padrões arquiteturais definem estruturas de projeto arquitetural (soluções arquiteturais) **bem estabelecidas** na prática e que podem ser **facilmente reutilizadas**.

Para isso, **descreve elementos arquiteturais** (e.g., módulos, componentes, e serviços) que compõem a solução, bem como suas **relações** (dependência, colaboração, controle), **propriedades** (responsabilidades, funcionalidades) e **restrições** (comunicação unidirecional, bidirecional, eventos, mensagens)

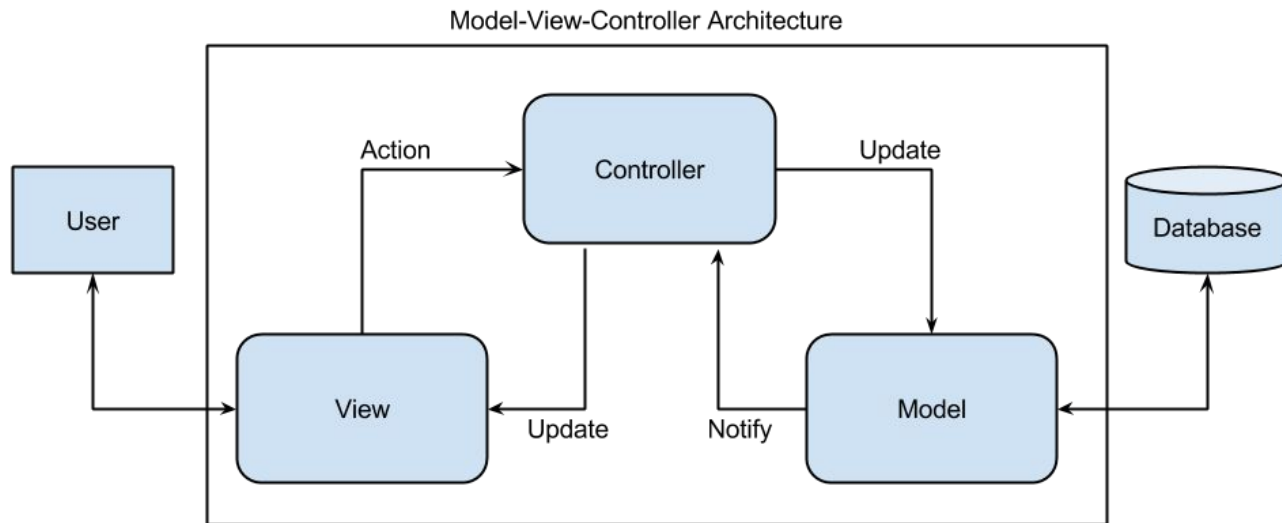
Exemplos de Padrões Arquiteturais

- Camadas (Layers)
- Pipes e Filtros (Pipes-and-Filters)
- Cliente-Servidor (Client-Server)
- Modelo-Visão-Controlador (Model-View-Controller – MVC)
- Arquitetura Orientada a Serviços (Service Oriented Architecture – SOA)
- Microserviços (Microservices)
- Publish-Subscribe
- Data Model
- Blackboard
- Broker
- Barramento (Bus)
- Shared Data
- Peer-to-Peer
- Blockchain
- ...

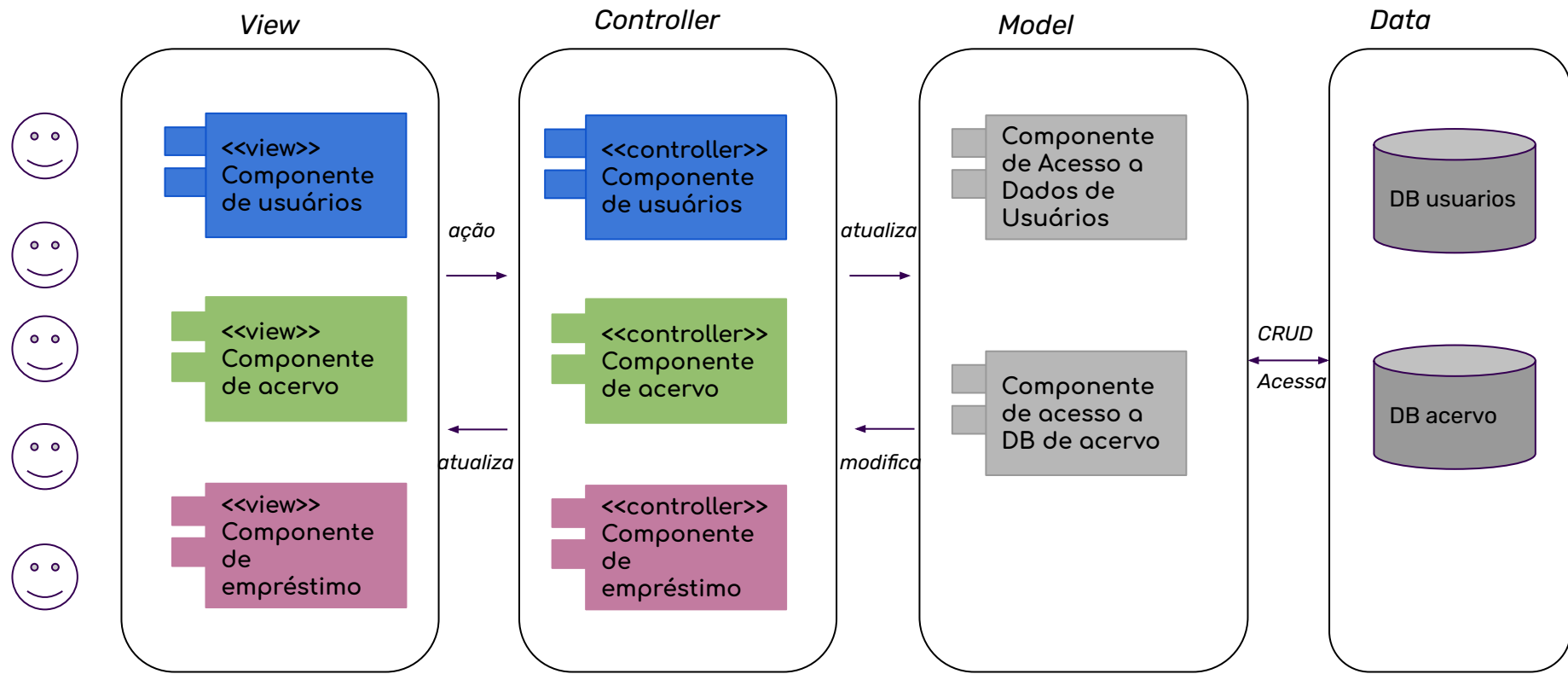
Recomendado: <https://www.developertoarchitect.com/lessons/>

Aplicando o Padrão MVC

MVC (Model-View-Controller)



Aplicando o Padrão MVC no Sistema de Biblioteca



Abstrações nos Elementos de Projeto

Sistema

Componentes

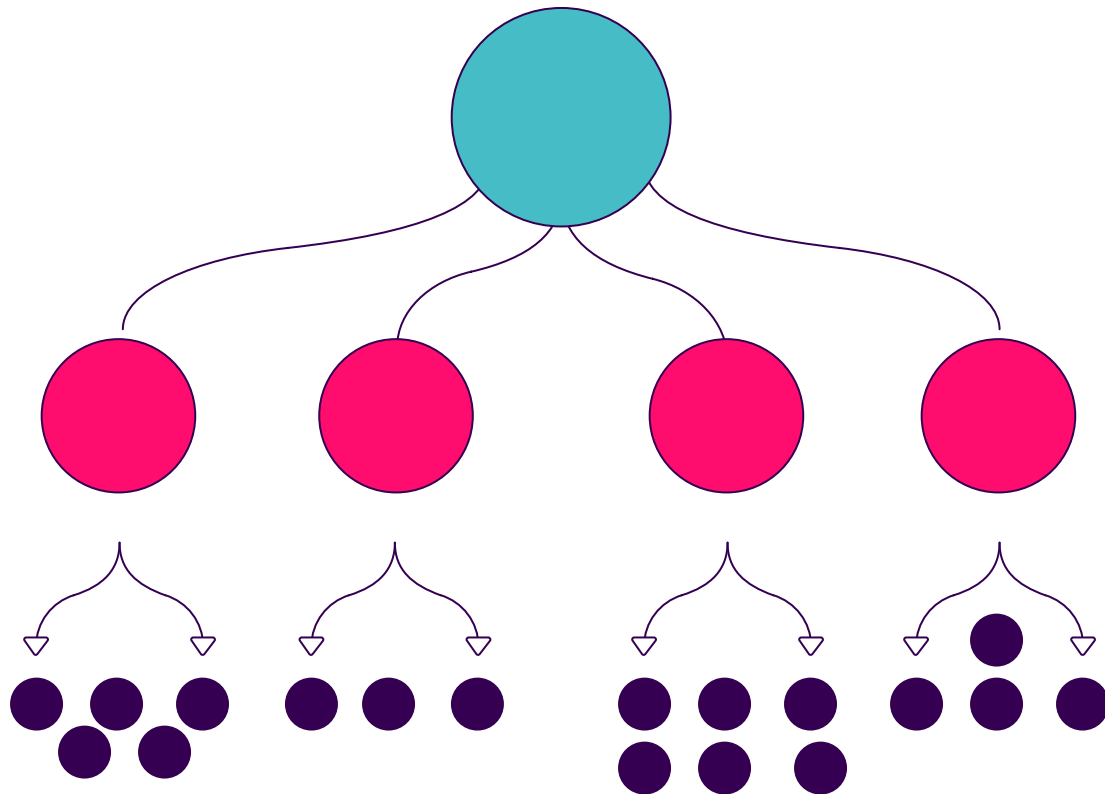
(Nível de Arquitetura -
Projeto Abstrato)

PADRÕES DE ARQUITETURA

Classes e objetos

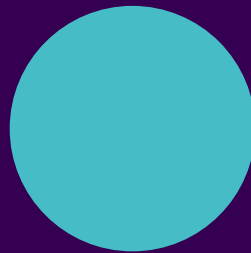
(Nível interno de Componente -
Projeto Concreto)

PADRÕES GoF



04

Identificação de Elementos Concretos (Classes e Objetos)



Abstrações nos Elementos de Projeto

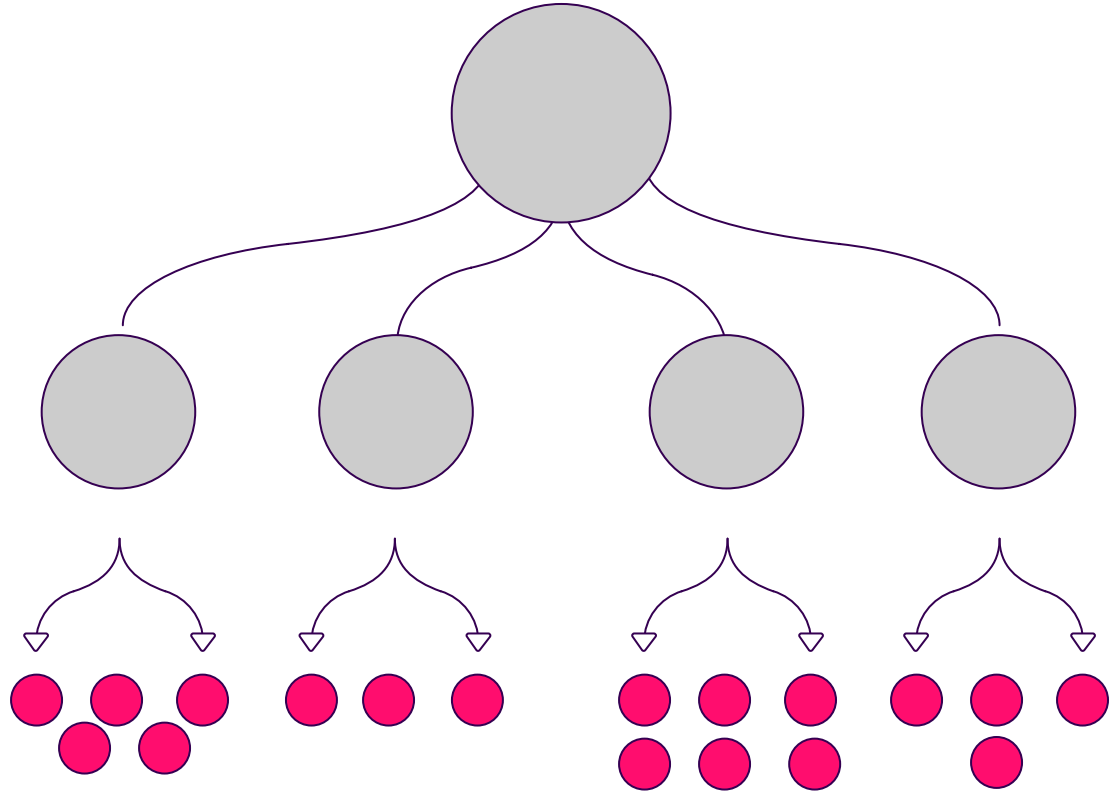
Sistema

Componentes

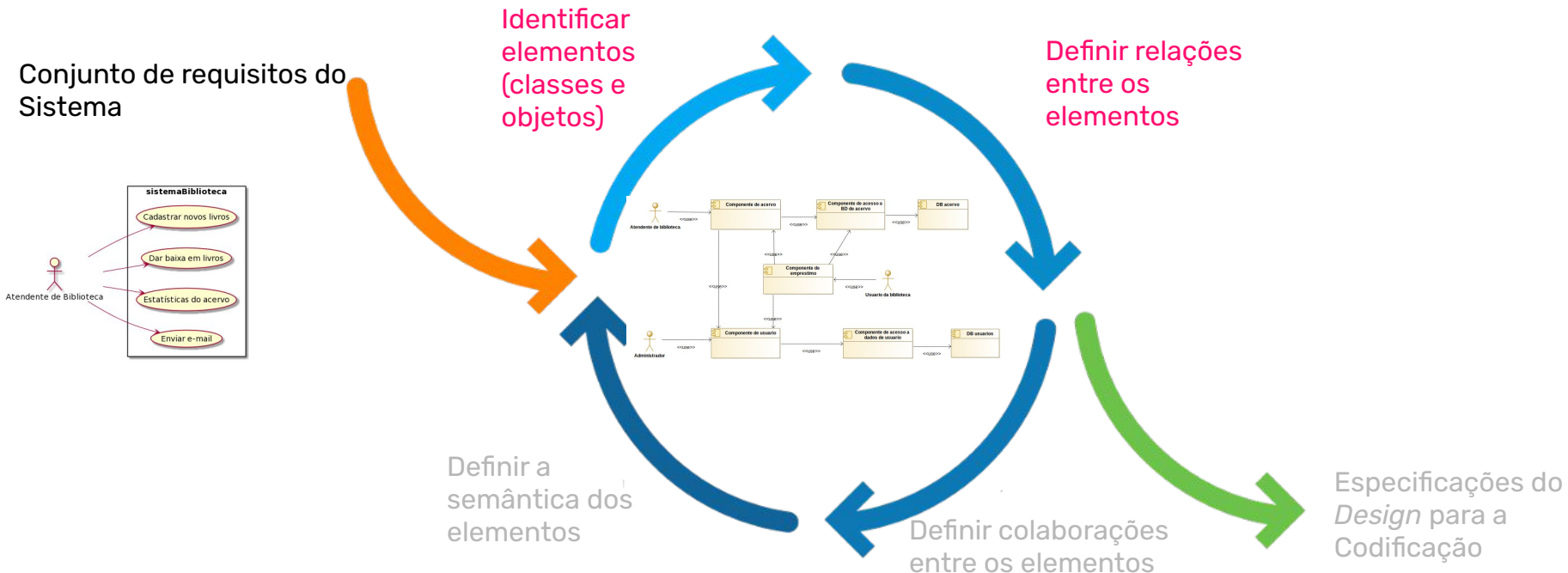
*(Nível de Arquitetura -
Projeto Abstrato)*

Classes e objetos

*(Nível interno de Componente -
Projeto Concreto)*



O Subprocesso de Análise e Projeto (Design)



Identificação de Classes dos Componentes

Com base nas responsabilidades do componente, identificar possíveis classes e objetos relacionados a:

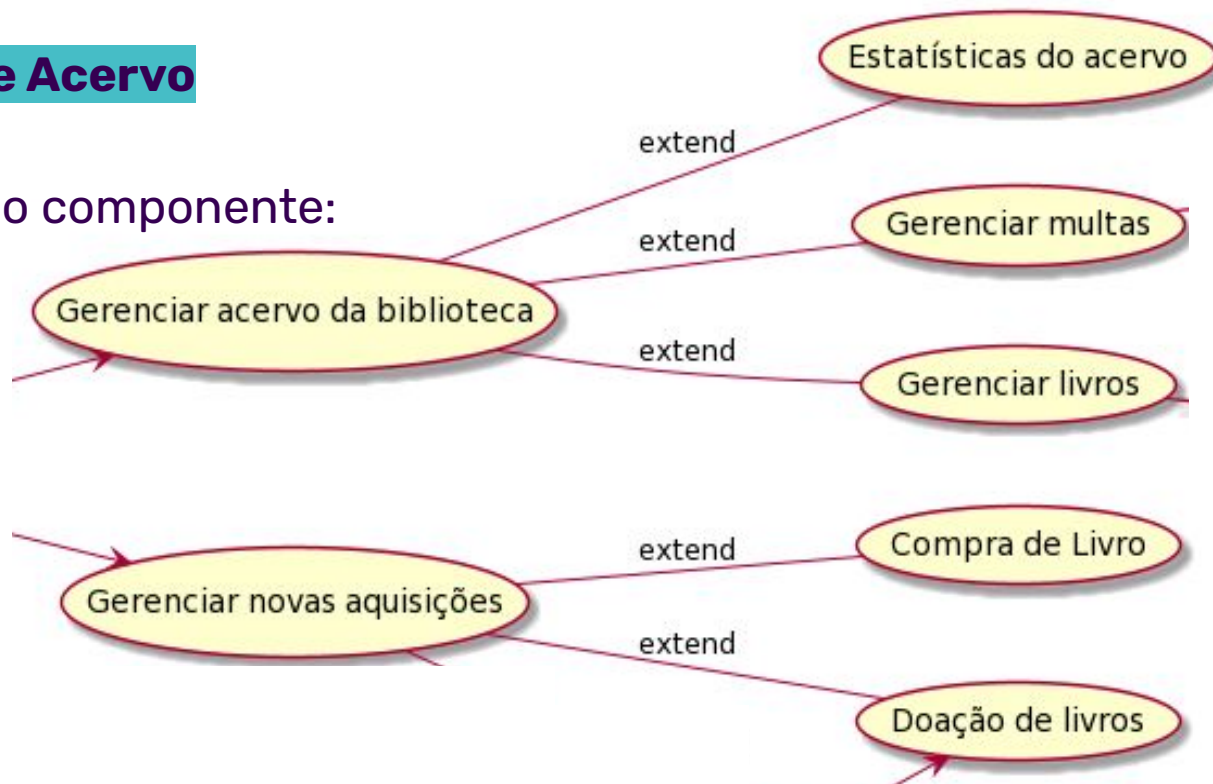
- Coisas tangíveis
- Perfis de usuários
- Eventos ou ações
- Interações
- Lugares
- Organizações
- Conceitos
- Transações

Identificação de Classes dos Componentes

Exemplo: Componente de Acervo

UC sob responsabilidade do componente:

- **Coisas tangíveis** →
- **Perfis de usuários** →
- **Eventos ou ações** →
- **Lugares** →
- **Organizações** →
- **Conceitos** →
- **Transações** →

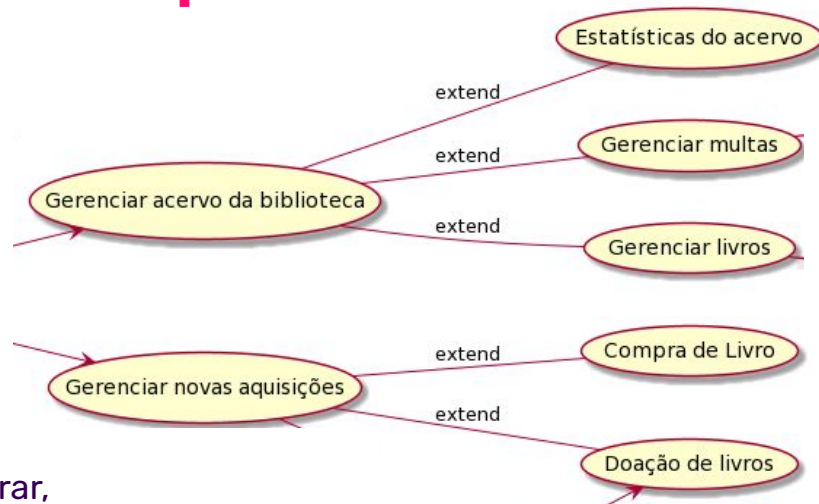


Identificação de Classes dos Componentes

Exemplo: Componente de Acervo

UC sob responsabilidade do componente:

- **Coisas tangíveis** → livro
- **Perfis de usuários** → atendente, administrador
- **Eventos ou ações** → gerenciar, adquirir, multar, comprar, doar, registrar
- **Lugares** → biblioteca, andar, corredor, prateleira, seção, recepção, administração → (implícito)
- **Organizações** → instituto, escola, universidade → (implícito)
- **Conceitos** → estatísticas, acervo, multa, aquisição
- **Transações** → Registro de doação, registro de compra, gerar estatísticas, calcular multa, ...



Identificação de Classes dos Componentes

Identificação de Classes:

Selecione aqueles elementos que se referem a **entidades que devem ser lembradas** pelo sistema: fazem algo, sabem algo, conhecem algo, têm dados necessários para executar as funcionalidades do sistema ...

Identificação de Classes dos Componentes

Identificação de Classes:

Selecione aqueles elementos que se referem a **entidades que devem ser lembradas** pelo sistema: fazem algo, sabem algo, conhecem algo, têm dados necessários para executar as funcionalidades do sistema ...

Quais dos seguintes elementos identificados para o componente acervo podem ser consideradas **classes** de objetos?

Livro, atendente, administrador, biblioteca, andar, corredor, prateleira, seção, recepção, administração, instituto, escola, universidade, gerenciar, adquirir, multar, comprar, doar, registrar, estatísticas, acervo, multa, aquisição, registro de doação, registro de compra

Identificação de Classes dos Componentes

Identificação de Classes:

Selecione aqueles elementos que se referem a **entidades que devem ser lembradas** pelo sistema: fazem algo, sabem algo, conhecem algo, têm dados necessários para executar as funcionalidades do sistema ...

Dos elementos identificados para o componente acervo, podemos identificar as seguintes classes (em cor verde):

- **Livro, atendente, administrador, biblioteca, localização do livro**, andar, corredor, prateleira, seção, recepção, administração, **instituto, escola, universidade**, gerenciar, adquirir, multar, comprar, doar, registrar, estatísticas, **acervo**, multa, aquisição, **registro de doação, registro de compra**

Identificação de Classes dos Componentes

Conceitos Canônicos

Representação padrão, única e correta de um dado ou modelo que serve como referência universal para evitar erros ou duplicações

Identificação de Classes dos Componentes

Refinamento de Classes:

Identifique sinônimos para agrupá-los em um **conceito canônico** que os represente melhor (melhorar coesão)

Dos elementos identificados para o componente acervo:

- Seção, andar, corredor, prateleira → localização
- Aquisição, compra, doação → aquisição
- Instituto, escola, universidade → unidade da universidade
- atendente, administrador → usuário
- Registro de doação, registro de compra → registro de aquisição

Identificação de Elementos de Componentes

Identificação de Propriedades ou Atributos das Classes de Elementos

(componente de acervo)

Classes de Elementos Identificadas	Possíveis atributos
• Livro • Usuário • Biblioteca • Localização • Unidade da universidade • Aquisição • Registro de aquisição	• ... • ... • ... • ... • ... • ... • ...

Identificação de Elementos de Componentes

Identificação de Propriedades ou Atributos das Classes de Elementos

(componente de acervo)

Classes de Elementos Identificadas	Possíveis atributos
• Livro • Usuário • Biblioteca • Localização • Unidade da universidade • Aquisição • Registro de aquisição	• Autor, título, ano de publicação, páginas, localização... • Nome, matrícula, email, telefone ... • Endereço • Andar, corredor, prateleira, número • Nome, endereço, bloco • Tipo de aquisição (ex. compra, doação) • Data, aquisição, usuário responsável da aquisição, livro adquirido ...



Relações entre os Elementos (Classes) no Nível de Componente



Identificação de Relações entre Classes

Tipos de relações entre elementos:

- Associação
- Herança simples e herança múltipla
- Agregação
- Composição



Relacionamento entre classes do tipo:

ASSOCIAÇÃO

Identificação de Relações entre Elementos

- Se dois elementos (objetos) devem colaborar então precisa haver uma relação do tipo **ASSOCIAÇÃO** entre eles

Classes de objetos identificadas	Possíveis atributos
• Livro • Usuário • Biblioteca • Localização • Unidade da universidade • Aquisição • Registro de aquisição	• Autor, título, ano de publicação, páginas, ID, localização... • Nome, matrícula, email, telefone ... • Endereço • Andar, corredor, prateleira, número • Nome, endereço • Tipo de aquisição (ex. compra, doação) • Data, aquisição, usuário responsável da aquisição, livro adquirido ...

**Atributos destacados podem ser considerados objetos*

Identificação de Associações entre Elementos

Identificar possíveis associações entre elementos

Exemplos de Possíveis Associações:

- Ter, ser, fazer, estar, comprar, registrar, usar, emprestar, reservar, devolver, desativar, modificar, multar, fazer parte de, compor ...

Livro **tem** autor(es)

Livro **tem** localização

Livro **faz parte** do acervo

Biblioteca **tem** acervo

Biblioteca **pertence a** unidade da universidade

Registro de aquisição **é** de um tipo de aquisição

Registro de aquisição **é** realizado por um (usuário) responsável

Registro de aquisição **cataloga** um novo livro ...

Identificação de Associações entre Elementos

Definir a multiplicidade das associações entre elementos

Exemplos

UM Livro **tem** **UM OU MAIS** autor(es)

UM Livro **tem** **EXATAMENTE UMA** localização

UM Livro **faz parte** de **EXATAMENTE UM** acervo

UMA Biblioteca **tem** **EXATAMENTE UM** acervo

UMA Biblioteca **pertence** **EXATAMENTE A UMA** unidade da universidade

UM Registro de aquisição **é** de **EXATAMENTE UM** tipo de aquisição

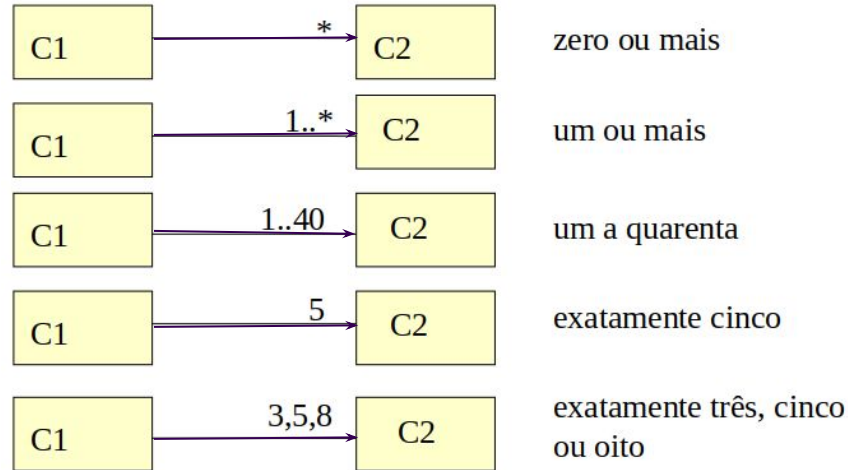
UM Registro de aquisição **é** realizado por **EXATAMENTE UM** (usuário) responsável

UM Registro de aquisição **cataloga** **EXATAMENTE UM** novo livro ...

Identificação de Associações entre Elementos

Definir a multiplicidade das associações entre elementos

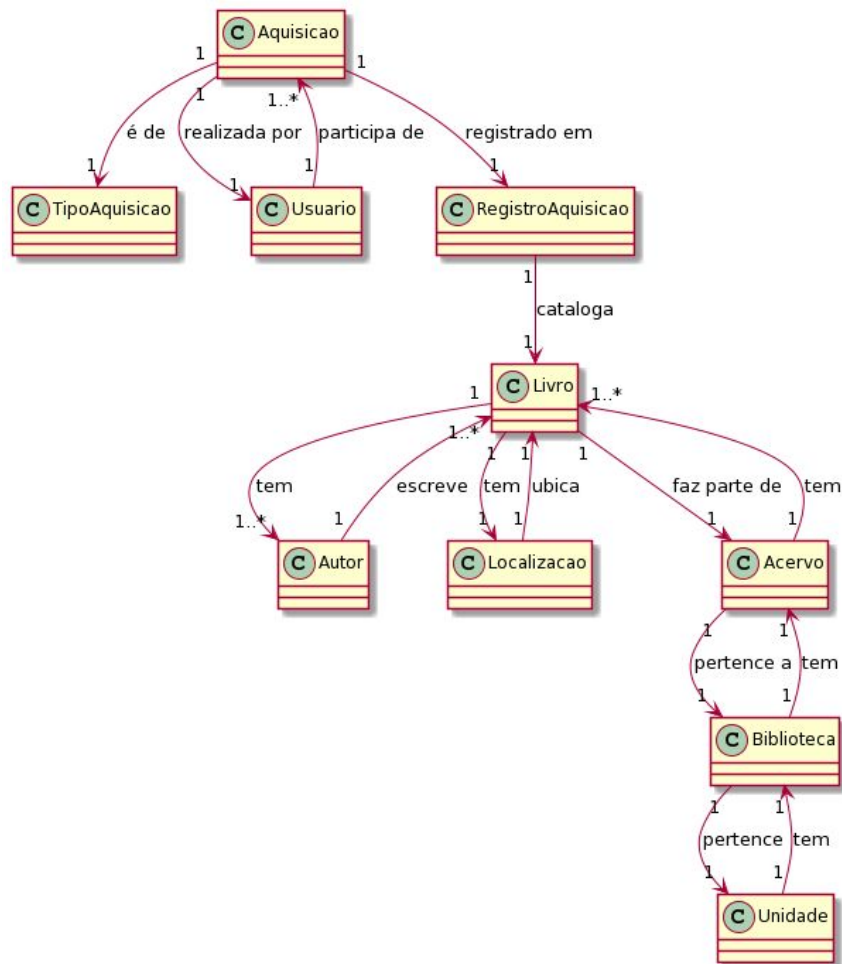
Graficamente é representado como:



Identificação de Associações entre Elementos

Modelo de classes UML do componente Acervo

Multiplicidade das associações entre classes de elementos





Relacionamento entre classes do tipo:

HERANÇA

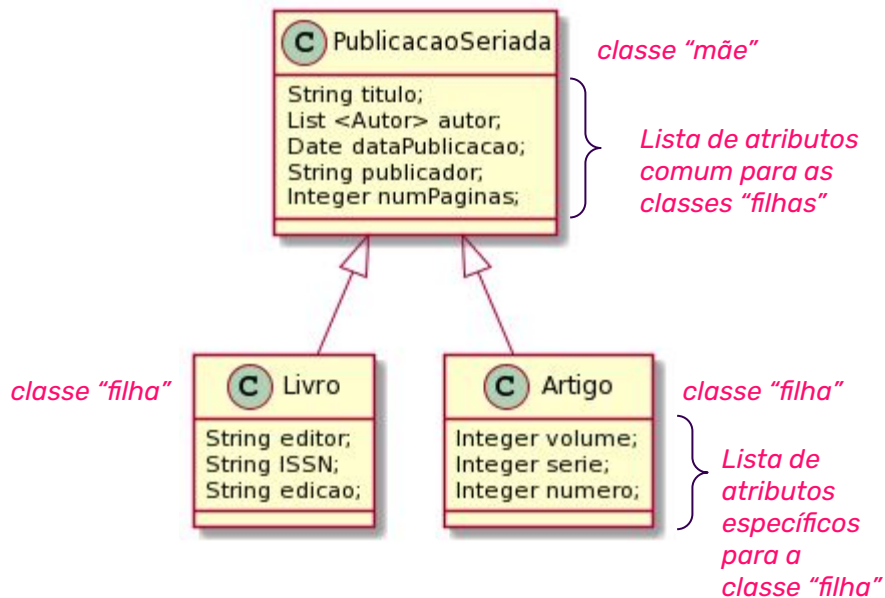
Relação do Tipo: Herança

- Se apresenta quando dois elementos têm alta similaridade mas também apresentam importantes diferenças.

*Lista de atributos
tipada*



Relação do Tipo: Herança

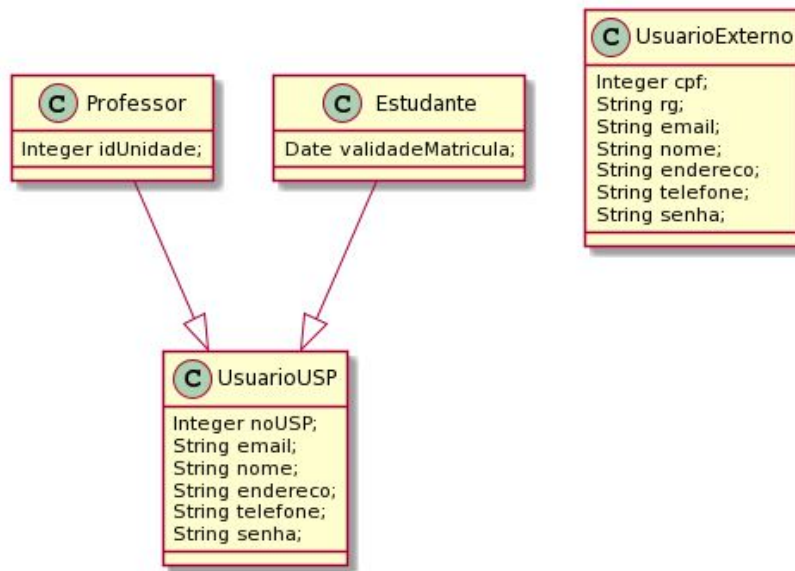


- As classes de elementos derivadas (ou classes filhas) **herdam** atributos da superclasse (ou classe mãe)
- Superclasses são generalizações ou abstrações
- As classes derivadas são mais específicas ou concretas
- Símbolo de herança: 
- Leitura: **é um** , **herda de**

Livro **é uma** publicacaoSeriada
Artigo **é uma** publicacaoSeriada

Relação do Tipo: Herança

- Analisemos a seguinte situação:

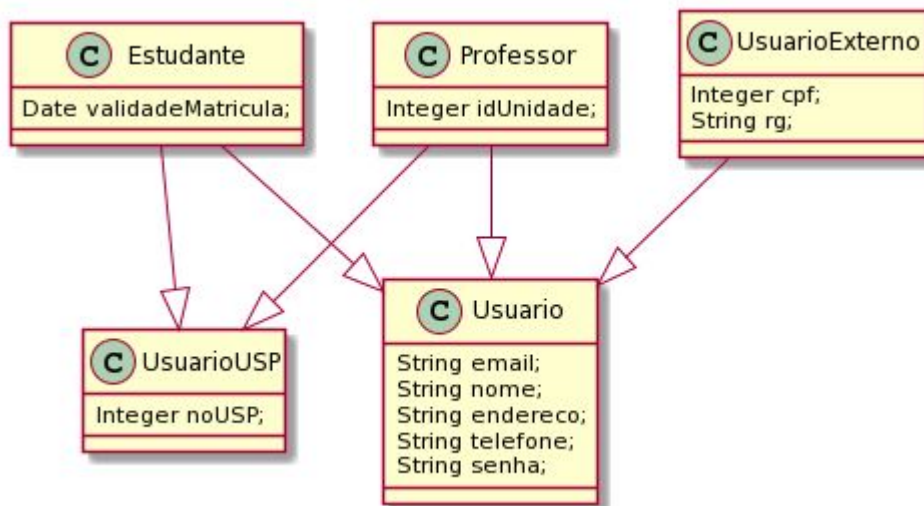


*Atributos semelhantes ao elemento **UsuarioUSP**, porém um **UsuarioExterno** não possui noUSP.*

Relação de Herança não pode ser possível neste caso!

Relação do Tipo: Herança Múltipla

- Através da herança múltipla é possível combinar as características de várias superclasses de elementos existentes como um ponto de partida para a definição de uma nova classe.
- Herança múltipla evita a duplicação de atributos comuns a várias classes





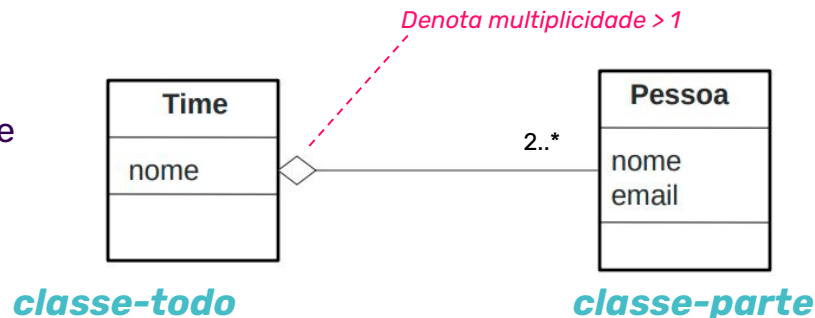
Relacionamento entre classes do tipo:

AGREGAÇÃO

Relação do Tipo: Agregação

- Tipo especial de associação
 - Demonstra que as informações de uma classe de elementos precisam ser complementadas por outra classe
 - A classe que agrega outras classes é chamada de **classe-todo**
 - A classe que complementa uma outra classe é chamada de **classe-parte**
 - A classe-parte é útil para o sistema sendo ou não parte da **classe-todo**
- Por exemplo: Um time é um agregado de um conjunto de pessoas
 - Símbolo: 
 - Leitura: **faz parte de, precisa de**

No mínimo 2 pessoas **fazem parte de** no mínimo um time
Um time **precisa de** no mínimo 2 pessoas



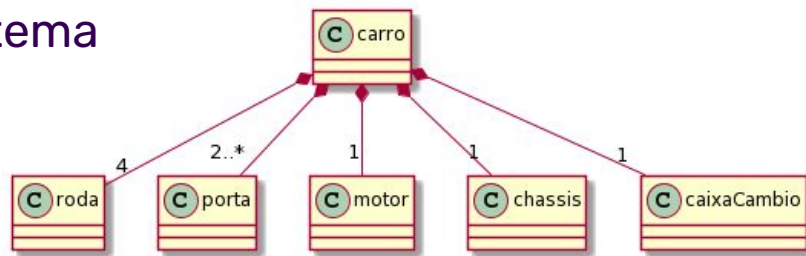


Relacionamento entre classes do tipo:

COMPOSIÇÃO

Relação do Tipo: Composição

- Uma variação do tipo agregação
 - Representa um vínculo mais forte entre classes-todo e classes-parte
 - Classes-parte têm que pertencer à classe-todo
 - O todo não existe (ou não faz sentido) sem as partes
 - Ou, as partes não existem (ou não faz sentido) sem o todo
- Por exemplo: Um carro é o resultado da agregação de elementos como rodas, motor, chassis, caixa de cambio, portas, etc.
- O carro não existe sem suas partes
- Cada parte isolada é inútil para o sistema
 - Símbolo: 
 - Leitura: **é composto de**



Um carro **é composto de** exatamente 4 rodas

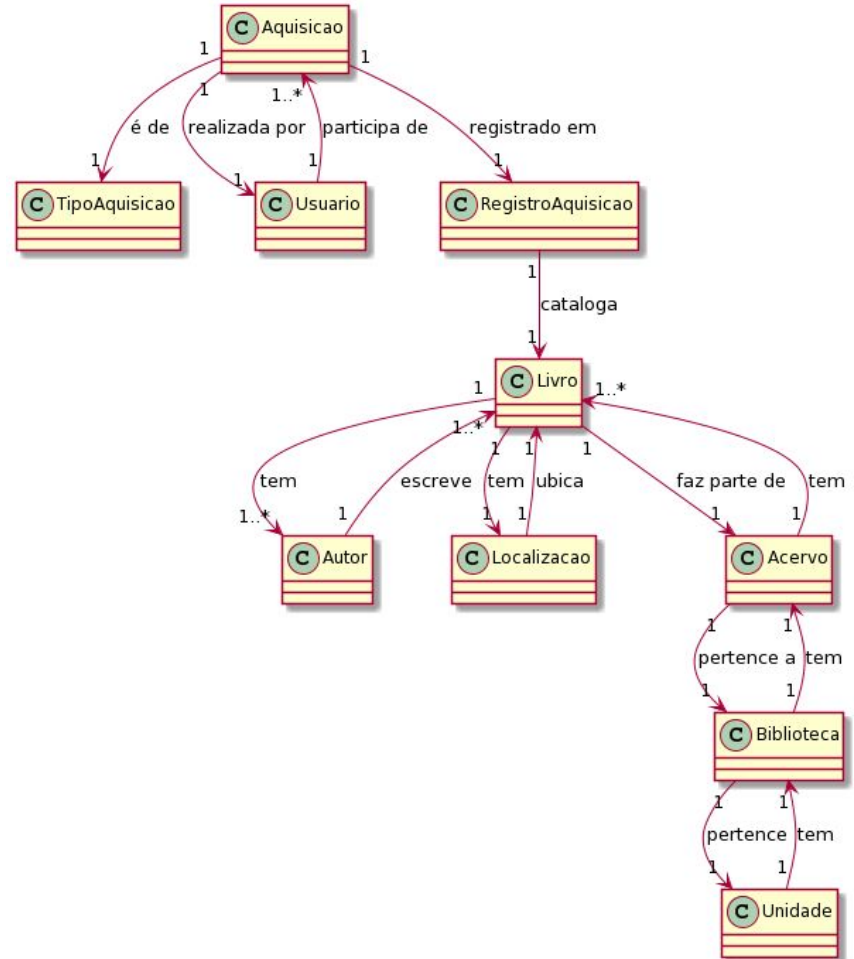
Um carro **é composto de** no mínimo 2 portas

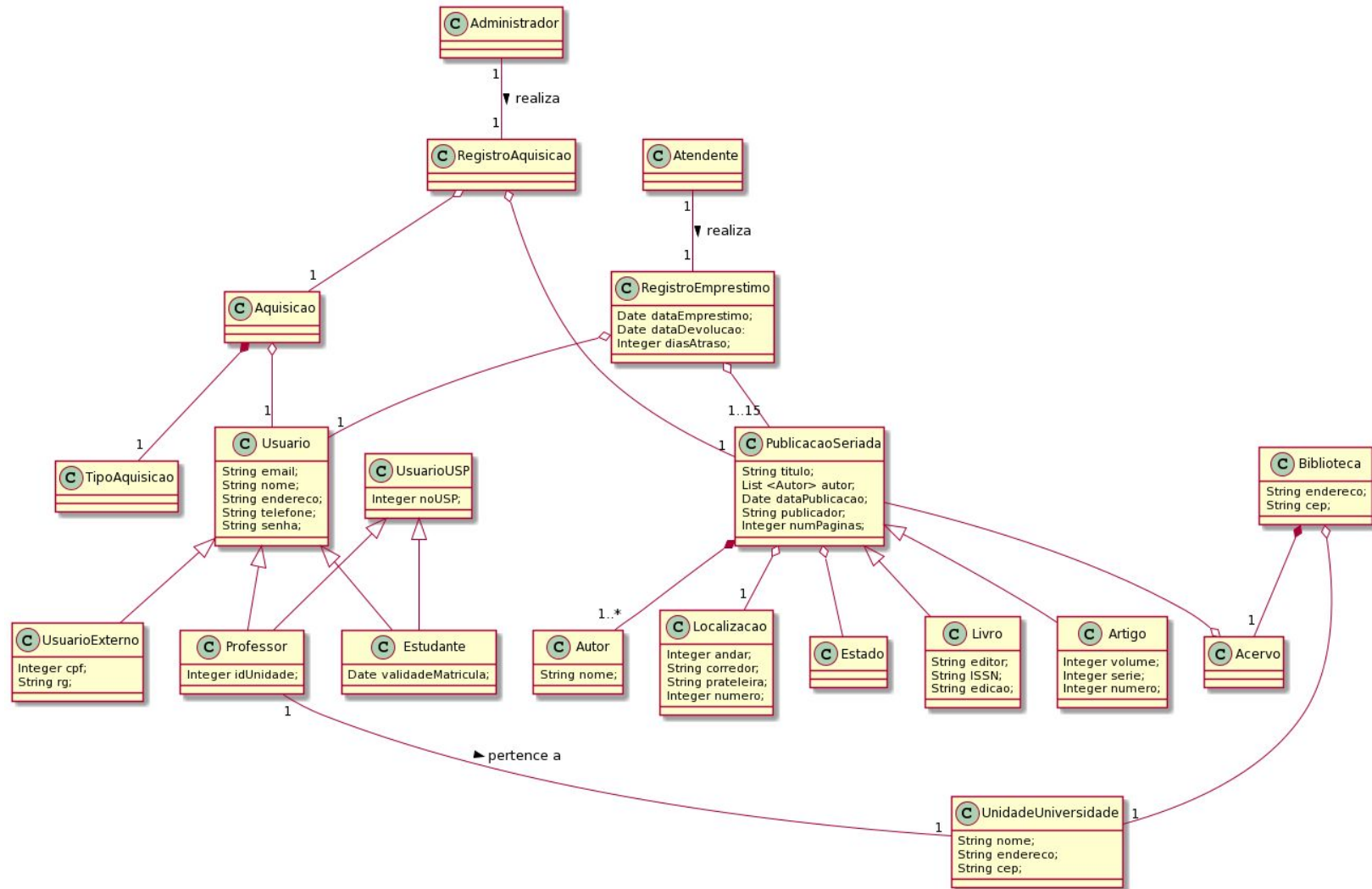
Refinando Associações

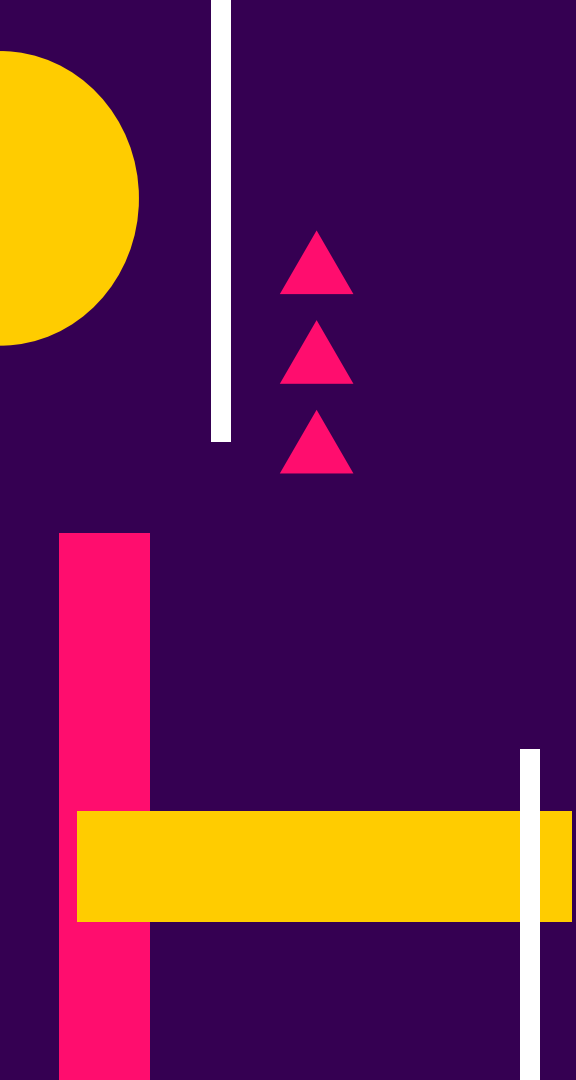
Das associações identificadas para as classes do componente de Acervo quais são do tipo:

- Agregação?
- Composição?

Quais generalizações (heranças) são necessárias?





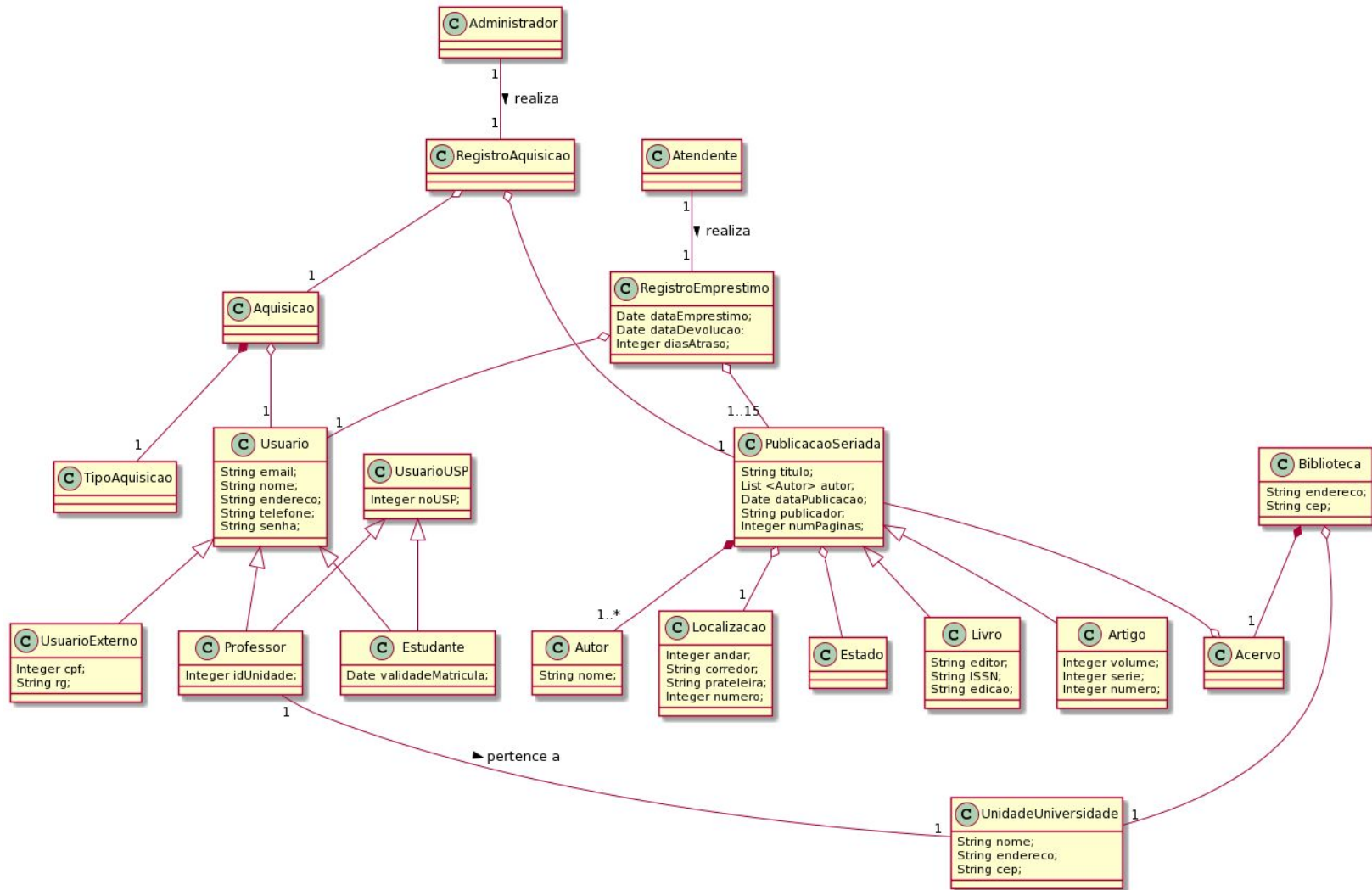


05

Modelo Conceitual do Sistema

Modelo Conceitual do Sistema

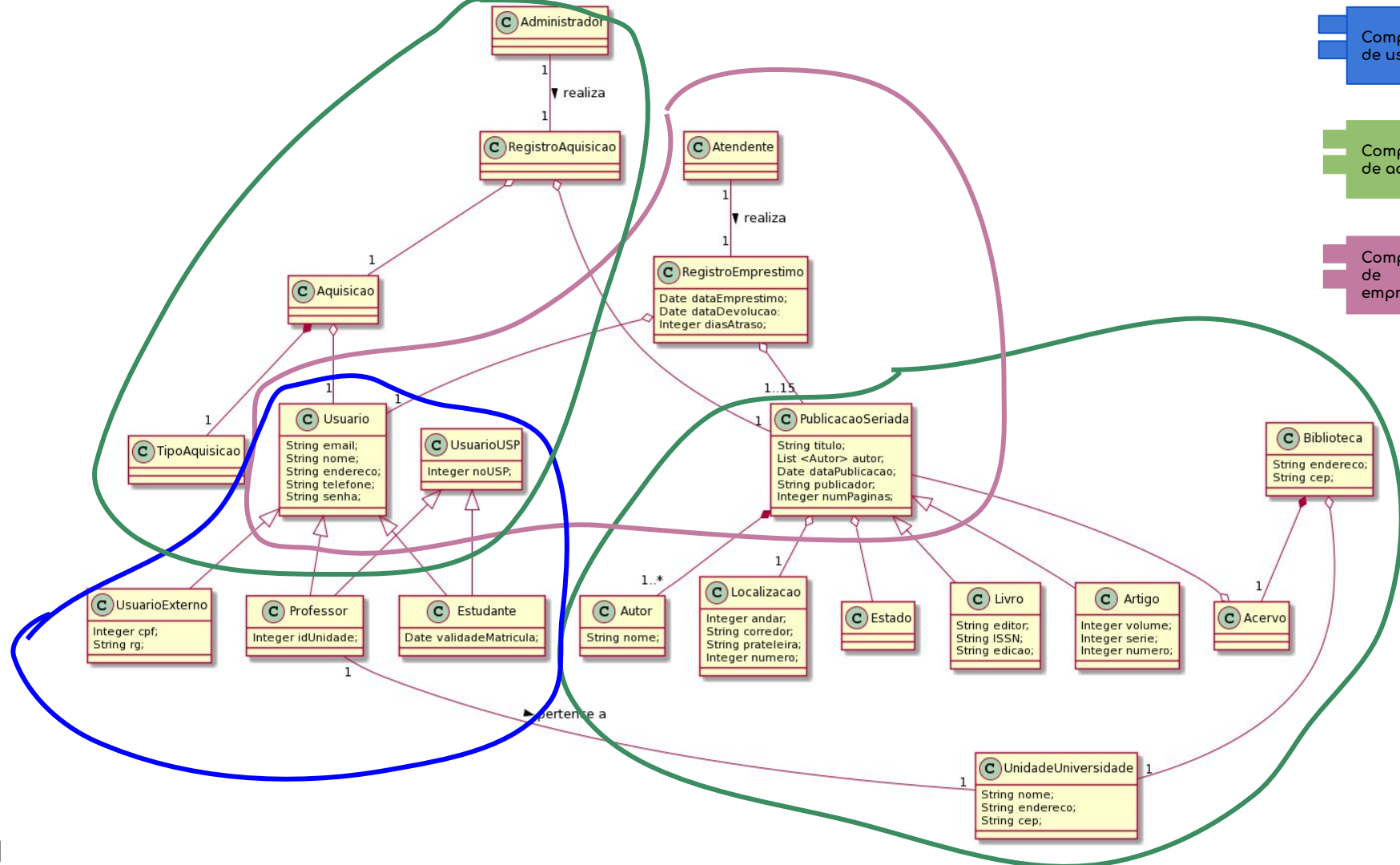
- Modelo abstrato do sistema
- Mostra todos os elementos (conceitos) importantes no domínio do sistema, bem como as associações entre esses conceitos
 - A junção dos modelos de elementos e associações de cada componente do sistema, estaremos representando o modelo conceitual do sistema
 - A junção deve manter a **integridade e alta coesão** entre os elementos
- Ajuda a esclarecer a terminologia ou vocabulário do domínio
- É o primeiro modelo para ter uma idéia do **tamanho e complexidade** do sistema em relação à quantidade de elementos e relações entre eles
- Pode ser utilizado como base para a definição do **modelo de entidade-relacionamento (MER)** durante o projeto do **banco de dados** do sistema
- É a base para as abordagens de desenvolvimento orientado a modelos (MDD - Model Driven Development)



Componente de usuários

Componente de acervo

Componente de empréstimo



Referências

Capítulo 6. Process. Livro: Grady Booch, robert A. Maksymchuk, Michael W. Engle, Bobbi J. Young, Jin Conallen, Kelli A.Houston. Object-Oriented Analysis and Design with Applications. 3rd Ed. 2007.

Capítulo 4. Process. Livro: Grady Booch, robert A. Maksymchuk, Michael W. Engle, Bobbi J. Young, Jin Conallen, Kelli A.Houston. Object-Oriented Analysis and Design with Applications. 3rd Ed. 2007.

Martin Fowler. 2015. Yagni. <https://www.martinfowler.com/bliki/Yagni.html>

Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal. 1996. Pattern-Oriented Software Architecture - Volume 1: A System of Patterns. Wiley Publishing.

Capítulo 3. 2015. Brahma Dathan and Sarnath Ramnath. Object Oriented Analysis, Design and Implementation. An Integrated Approach. Springer. Pp. 479.